

Flutter Project Documentation: Clean Architecture with Modules

Overview

This Flutter project follows Clean Architecture principles, modular design, and leverages OpenAPI generated services/models. The project is organized to separate concerns, improve scalability, testability, and maintainability.

Key points:

- Modules instead of features for clarity
- Domain-driven design (DDD) principles
- Generated code isolated for safe regeneration
- Dependency Injection ready
- Supports Bloc / Provider / Riverpod for state management

Folder Structure

```
lib/  
├─ core/           # Shared utilities and base classes  
├─ generated/      # OpenAPI generated services & models  
├─ modules/        # App modules (Auth, Reservation, Marketplace,  
Forums)  
├─ config/         # App configuration (routes, theme, environment)  
├─ injection_container.dart # Dependency injection setup  
└─ main.dart       # Entry point
```

core/ – Shared Resources

Contains reusable elements across all modules.

```
lib/core/  
├─ errors/         # Failure classes, custom exceptions  
├─ network/        # API/network helpers  
├─ constants/      # App constants  
├─ utils/          # Utility functions, formatters, validators  
└─ widgets/        # Shared widgets (buttons, loaders)
```

generated/ – OpenAPI Code

Contains all generated API clients and models.

```
lib/generated/
├─ api/          # Generated API service classes
├─ models/       # Generated DTO models
└─ serializers/ # Generated serialization helpers
```

Guidelines: - Do not manually modify this folder - Regenerate safely when API updates - Use in repositories to implement domain interfaces

modules/ – Business Modules

Each module contains Domain and Presentation layers.

```
lib/modules/
├─ auth/
├─ reservation/
├─ marketplace/
└─ forums/
```

Structure inside a module:

```
module_name/
├─ domain/          # Pure business logic
│   └─ entities/    # Business objects
│       └─ repositories/ # Abstract repositories (interfaces)
│           └─ usecases/ # Usecases / interactors
└─ presentation/    # UI layer
    ├── pages/      # Screens
    ├── blocs/      # State management (Bloc/Cubit/Provider)
    └─ widgets/     # UI components specific to module
```

config/ – Application Configuration

```
lib/config/
├─ routes/          # Named routes or GoRouter config
├─ theme/           # Light/Dark themes
├─ environment/     # Environment variables (dev, staging, prod)
└─ localization/    # Strings and translations
```

injection_container.dart – Dependency Injection

Central DI setup using `get_it` or other DI library.

main.dart – Entry Point

Contains the main app startup and routing configuration.

Best Practices

1. Modules = features: Keep them isolated and independent.
2. Generated code stays in generated/ only.
3. Domain layer is pure Dart (no Flutter, no API).
4. Repositories map generated models to entities.
5. Usecases are your application's business actions.
6. State management goes in presentation (Bloc / Riverpod / Provider).
7. DI centralizes all dependencies for easier testing.
8. Config folder holds everything environment-related.
9. Tests mirror the structure in test/.
10. Keep UI thin; all business logic is in domain/usecases.

Summary

- core/ → reusable tools
- generated/ → OpenAPI generated code
- modules/ → isolated business domains
- config/ → app setup and environment
- injection_container.dart → DI setup
- main.dart → Flutter app entry point